

Digital Image Processing

ASSIGNMENT -6

Perumalla Dharan

AP21110010201

The basic idea of frequency domain filtering is shown in the block diagram given below.

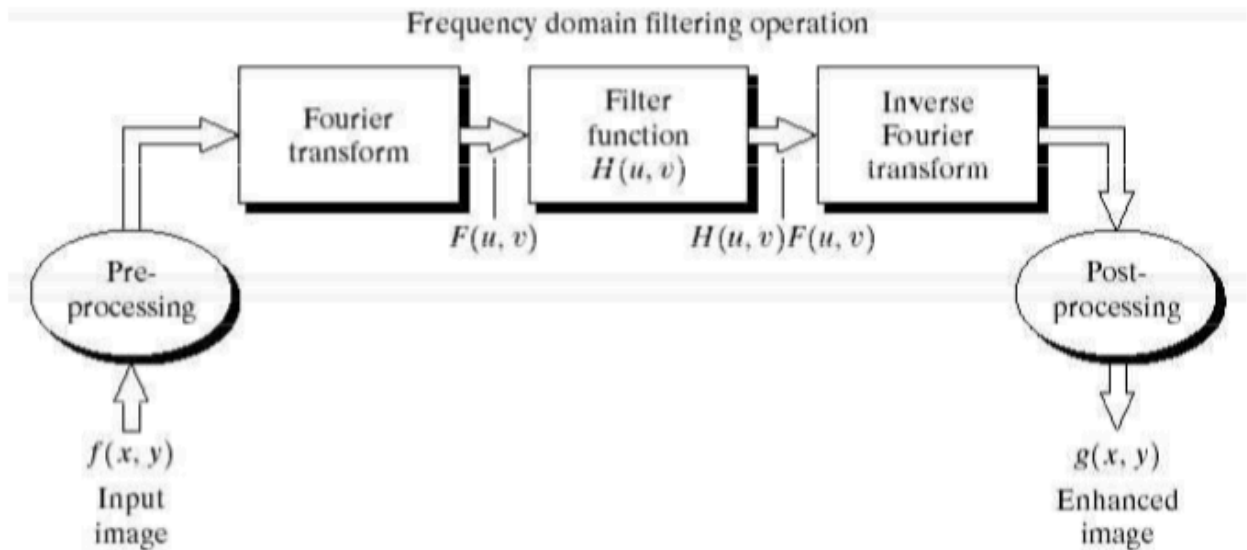


FIGURE 4.5 Basic steps for filtering in the frequency domain.

1. Develop programs to implement frequency domain smoothing filters (Ideal, Butterworth and Gaussian) and apply these filters on a gray scale image.
 - a. Compare/comment on the output of Ideal, Butterworth and Gaussian Low pass Filters having the same radii (cutoff frequency) value.
 - b. Consider a suitable gray scale image and demonstrate the ringing effect on the output of Ideal low pass frequency domain filter.
 - c. Compare the output of Butterworth low pass filters (order $n=2$) for different cut-off frequencies (5, 15, 30, 90, 120).
 - d. Compare the output of Gaussian low pass filters for different cut-off frequencies (5, 15, 30, 90, and 120).

```
import cv2
import numpy as np
```

```

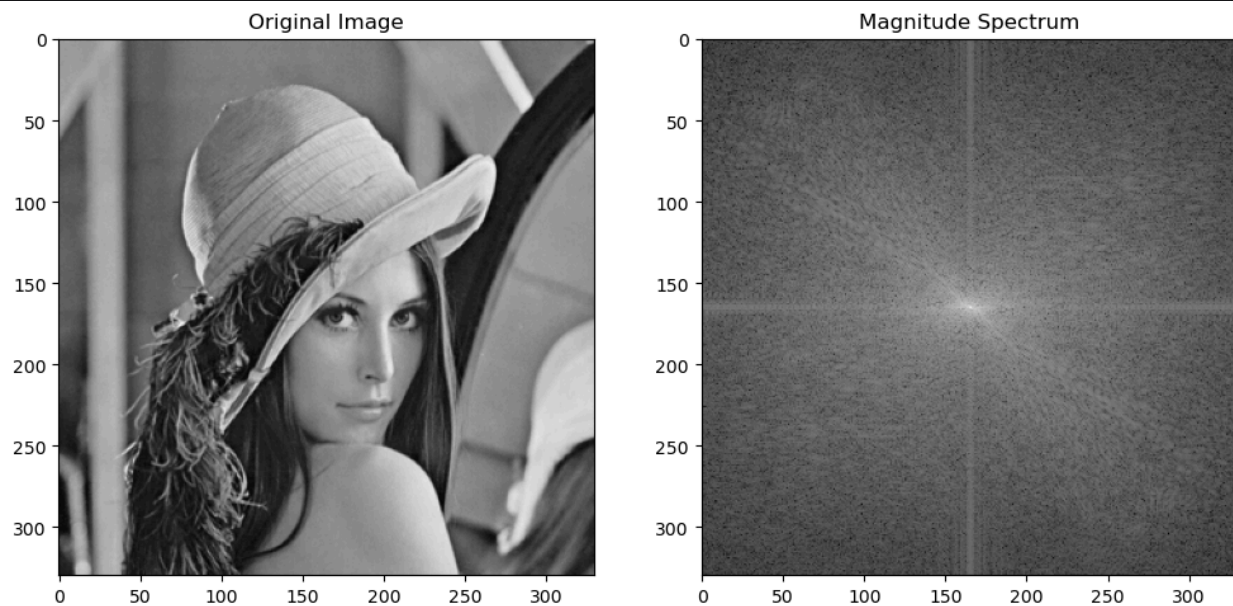
import matplotlib.pyplot as plt
from numpy.fft import fft2, fftshift, ifft2, ifftshift

img = cv2.imread(r"E:\SRM\Digital Image\Lab 6 - 14th
October\test_image.png", 0)

f_transform = fft2(img)
f_transform_shifted = fftshift(f_transform)
magnitude_spectrum = np.log(np.abs(f_transform_shifted) + 1)

plt.figure(figsize=(12,6))
plt.subplot(1,2,1), plt.imshow(img, cmap='gray'),
plt.title("Original Image")
plt.subplot(1,2,2), plt.imshow(magnitude_spectrum, cmap='gray'),
plt.title("Magnitude Spectrum")
plt.show()

```



```

# Ideal Low-Pass Filter
def ideal_low_pass_filter(shape, D0):
    M, N = shape

```

```

        U, V = np.meshgrid(np.arange(M), np.arange(N),
indexing='ij')

        D = np.sqrt((U - M/2)**2 + (V - N/2)**2)
        H = np.where(D <= D0, 1, 0).astype(np.float32)
        return H

# Butterworth Low-Pass Filter
def butterworth_low_pass_filter(shape, D0, n):
    M, N = shape
    U, V = np.meshgrid(np.arange(M), np.arange(N),
indexing='ij')
    D = np.sqrt((U - M/2)**2 + (V - N/2)**2)
    H = 1 / (1 + (D / D0) ** (2 * n)).astype(np.float32)
    return H

# Gaussian Low-Pass Filter
def gaussian_low_pass_filter(shape, D0):
    M, N = shape
    U, V = np.meshgrid(np.arange(M), np.arange(N),
indexing='ij')
    D = np.sqrt((U - M/2)**2 + (V - N/2)**2)
    H = np.exp(-(D ** 2) / (2 * (D0 ** 2))).astype(np.float32)
    return H

# Apply a filter and perform inverse Fourier Transform
def apply_filter_and_inverse(f_transform_shifted, H):
    Gshift = f_transform_shifted * H
    G = ifftshift(Gshift)
    g = np.abs(ifft2(G))
    return g

# Plotting function to compare original and filtered image
def plot_images(original, filtered, title):

```

```

plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(original, cmap='gray')
plt.title("Original Image")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(filtered, cmap='gray')
plt.title(title)
plt.axis('off')
plt.show()

# Set cutoff frequency
cutoff = 50

# Ideal Low-Pass Filter
H_ideal = ideal_low_pass_filter(img.shape, cutoff)
ideal_filtered_img = apply_filter_and_inverse(f_transform_shifted, H_ideal)
plot_images(img, ideal_filtered_img, f"Ideal Low Pass Filter (D0={cutoff})")

# Butterworth Low-Pass Filter (Order 2)
n = 2
H_butterworth = butterworth_low_pass_filter(img.shape, cutoff, n)
butterworth_filtered_img = apply_filter_and_inverse(f_transform_shifted, H_butterworth)
plot_images(img, butterworth_filtered_img, f"Butterworth Low Pass Filter (n={n}, D0={cutoff})")

# Gaussian Low-Pass Filter
H_gaussian = gaussian_low_pass_filter(img.shape, cutoff)

```

```
gaussian_filtered_img =  
apply_filter_and_inverse(f_transform_shifted, H_gaussian)  
plot_images(img, gaussian_filtered_img, f"Gaussian Low Pass  
Filter (D0={cutoff})")
```

Original Image



Ideal Low Pass Filter ($D_0=50$)



Original Image



Butterworth Low Pass Filter ($n=2$, $D_0=50$)



Original Image



Gaussian Low Pass Filter (D0=50)



```
# Apply Ideal Low-Pass Filter with ringing effect demonstration
cutoff = 30 # Sharp cut-off frequency to highlight ringing
H_ideal_ringing = ideal_low_pass_filter(img.shape, cutoff)
ideal_filtered_img_ringing =
apply_filter_and_inverse(f_transform_shifted, H_ideal_ringing)
plot_images(img, ideal_filtered_img_ringing, f"Ideal Low Pass
Filter - Ringing Effect (D0={cutoff})")
```

Original Image



Ideal Low Pass Filter - Ringing Effect (D0=30)



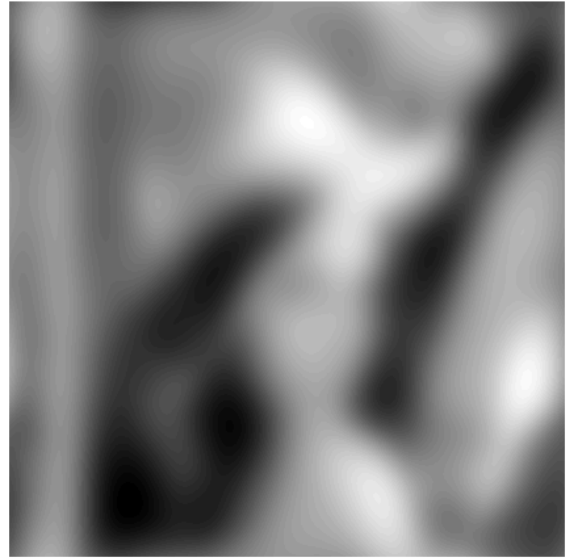
```
# Compare Butterworth Low-Pass Filters for different cutoff
frequencies
cutoffs = [5, 15, 30, 90, 120]
n = 2 # Fixed order of the filter

for c in cutoffs:
    H_butterworth = butterworth_low_pass_filter(img.shape, c, n)
    butterworth_filtered_img =
apply_filter_and_inverse(f_transform_shifted, H_butterworth)
    plot_images(img, butterworth_filtered_img, f"Butterworth Low
Pass Filter (n={n}, D0={c})")
```

Original Image



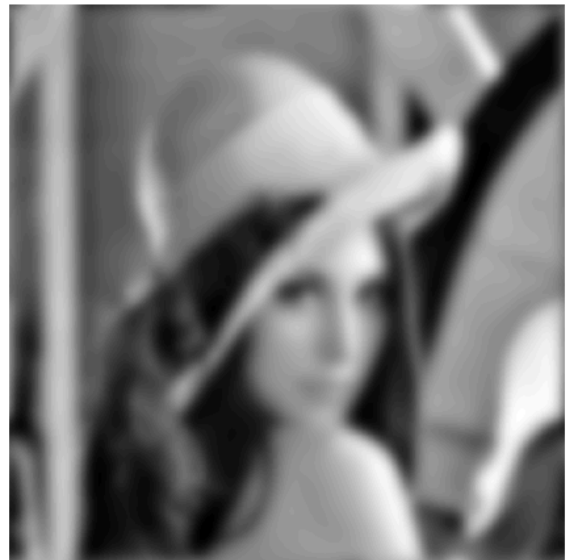
Butterworth Low Pass Filter ($n=2$, $D_0=5$)



Original Image



Butterworth Low Pass Filter ($n=2$, $D_0=15$)



Original Image



Butterworth Low Pass Filter ($n=2$, $D_0=30$)



Original Image



Butterworth Low Pass Filter (n=2, D0=90)



Original Image



Butterworth Low Pass Filter (n=2, D0=120)



```
# Compare Gaussian Low-Pass Filters for different cutoff
frequencies
cutoffs = [5, 15, 30, 90, 120]

for c in cutoffs:
    H_gaussian = gaussian_low_pass_filter(img.shape, c)
    gaussian_filtered_img =
apply_filter_and_inverse(f_transform_shifted, H_gaussian)
```

```
plot_images(img, gaussian_filtered_img, f"Gaussian Low Pass  
Filter (D0={c})")
```

Original Image



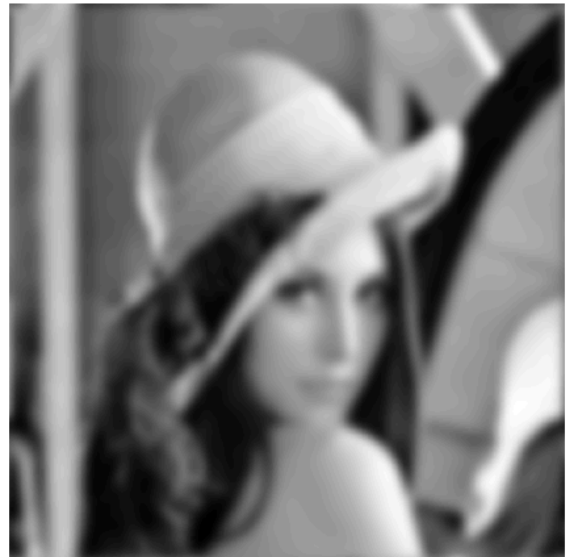
Gaussian Low Pass Filter (D0=5)



Original Image



Gaussian Low Pass Filter (D0=15)



Original Image



Gaussian Low Pass Filter ($D_0=30$)



Original Image



Gaussian Low Pass Filter ($D_0=90$)



Original Image



Gaussian Low Pass Filter ($D_0=120$)



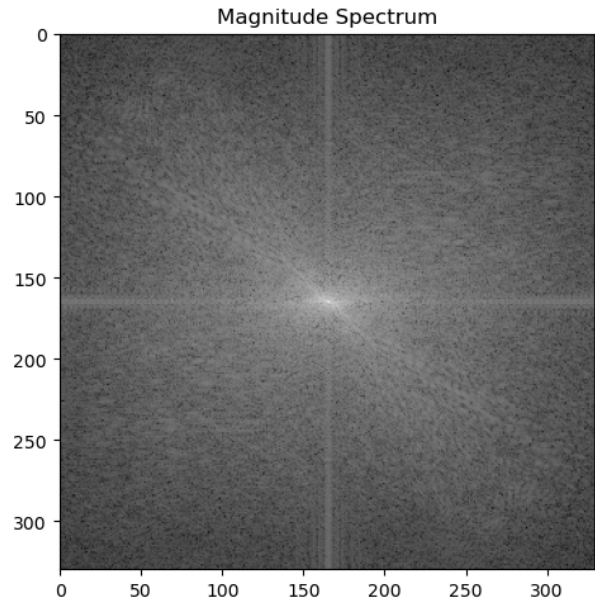
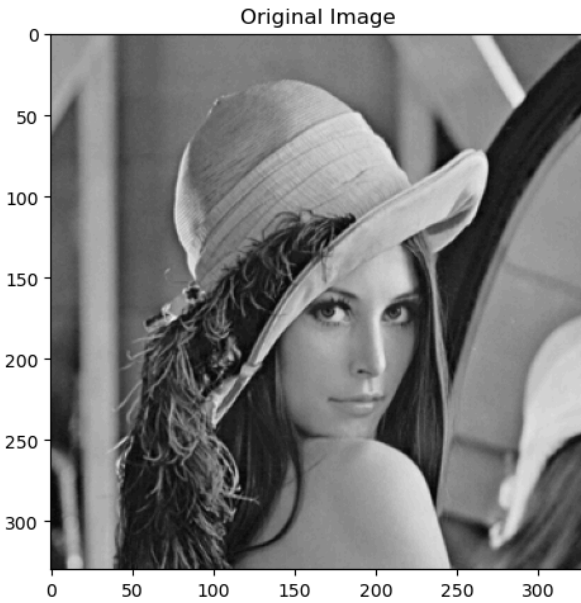
2. Develop programs to implement frequency domain sharpening/High pass filters (Ideal, Butterworth and Gaussian) and apply these filters on a gray scale image.
 - a. Compare/comment on the output of Ideal, Butterworth and Gaussian High pass Filters having the same radii (cutoff frequency) value.
 - b. Consider a suitable gray scale image and demonstrate the ringing effect on the output of Ideal high pass frequency domain filter.
 - c. Compare the output of Butterworth high pass filters (order $n=2$) for different cut-off frequencies (5, 15, 30, 90, 120).
 - d. Compare the output of Gaussian high pass filters for different cut-off frequencies (5, 15, 30, 90, and 120).

```
# Required Libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from numpy.fft import fft2, fftshift, ifft2, ifftshift

# Load the image (grayscale)
img = cv2.imread(r"E:\SRM\Digital Image\Lab 6 - 14th
October\test_image.png", 0)

# Fourier Transform
f_transform = fft2(img)
f_transform_shifted = fftshift(f_transform)
magnitude_spectrum = np.log(np.abs(f_transform_shifted) + 1)

# Display the original image and its Fourier Transform
plt.figure(figsize=(12,6))
plt.subplot(1,2,1), plt.imshow(img, cmap='gray'),
plt.title("Original Image")
plt.subplot(1,2,2), plt.imshow(magnitude_spectrum, cmap='gray'),
plt.title("Magnitude Spectrum")
plt.show()
```



```
# Ideal High-Pass Filter
def ideal_high_pass_filter(shape, D0):
    M, N = shape
    U, V = np.meshgrid(np.arange(M), np.arange(N),
indexing='ij')
    D = np.sqrt((U - M/2)**2 + (V - N/2)**2)
    H = np.where(D > D0, 1, 0).astype(np.float32)
    return H

# Butterworth High-Pass Filter
# def butterworth_high_pass_filter(shape, D0, n):
#     M, N = shape
#     U, V = np.meshgrid(np.arange(M), np.arange(N),
indexing='ij')
#     D = np.sqrt((U - M/2)**2 + (V - N/2)**2)
#     H = 1 / (1 + (D0 / D) ** (2 * n)).astype(np.float32)
#     return H

# RuntimeWarning: divide by zero encountered in divide, occurs
because in the expression (D0 / D),
# there might be some points in the frequency domain where D =
0. This happens at the center of the
```

```

# frequency domain (DC component), where the distance is zero.
# To fix this issue, you can avoid division by zero by adding a
small value (epsilon) to the
# denominator or handling the center frequency separately.

def butterworth_high_pass_filter(shape, D0, n):
    M, N = shape
    H = np.zeros((M, N), dtype=np.float32)
    epsilon = 1e-5 # Small value to avoid division by zero
    for u in range(M):
        for v in range(N):
            D = np.sqrt((u - M / 2) ** 2 + (v - N / 2) ** 2)
            if D == 0: # Handle the center (DC component)
                H[u, v] = 0
            else:
                H[u, v] = 1 / (1 + (D0 / (D + epsilon)) ** (2 *
n)) # Avoid zero in the denominator
    return H

# Gaussian High-Pass Filter
def gaussian_high_pass_filter(shape, D0):
    M, N = shape
    U, V = np.meshgrid(np.arange(M), np.arange(N),
indexing='ij')
    D = np.sqrt((U - M/2)**2 + (V - N/2)**2)
    H = 1 - np.exp(-(D ** 2) / (2 * (D0 **
2))) .astype(np.float32)
    return H

# Apply a filter and perform inverse Fourier Transform
def apply_filter_and_inverse(f_transform_shifted, H):

```

```

    Gshift = f_transform_shifted * H
    G = ifftshift(Gshift)
    g = np.abs(ifft2(G))
    return g

# Plotting function to compare original and filtered image
def plot_images(original, filtered, title):
    plt.figure(figsize=(12, 6))
    plt.subplot(1, 2, 1)
    plt.imshow(original, cmap='gray')
    plt.title("Original Image")
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.imshow(filtered, cmap='gray')
    plt.title(title)
    plt.axis('off')
    plt.show()

# Set cutoff frequency
cutoff = 50

# Ideal High-Pass Filter
H_ideal = ideal_high_pass_filter(img.shape, cutoff)
ideal_filtered_img = apply_filter_and_inverse(f_transform_shifted, H_ideal)
plot_images(img, ideal_filtered_img, f"Ideal High Pass Filter (D0={cutoff})")

# Butterworth High-Pass Filter (Order 2)
n = 2
H_butterworth = butterworth_high_pass_filter(img.shape, cutoff,
n)

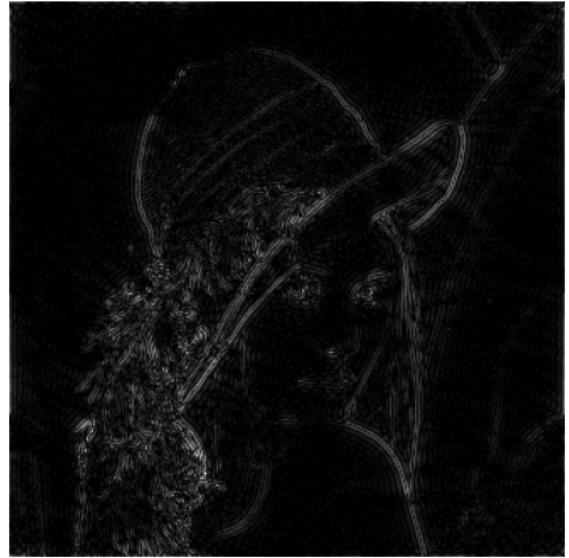
```

```
butterworth_filtered_img =  
apply_filter_and_inverse(f_transform_shifted, H_butterworth)  
plot_images(img, butterworth_filtered_img, f"Butterworth High  
Pass Filter (n={n}, D0={cutoff})")  
  
# Gaussian High-Pass Filter  
H_gaussian = gaussian_high_pass_filter(img.shape, cutoff)  
gaussian_filtered_img =  
apply_filter_and_inverse(f_transform_shifted, H_gaussian)  
plot_images(img, gaussian_filtered_img, f"Gaussian High Pass  
Filter (D0={cutoff})")
```

Original Image



Ideal High Pass Filter ($D_0=50$)



Original Image



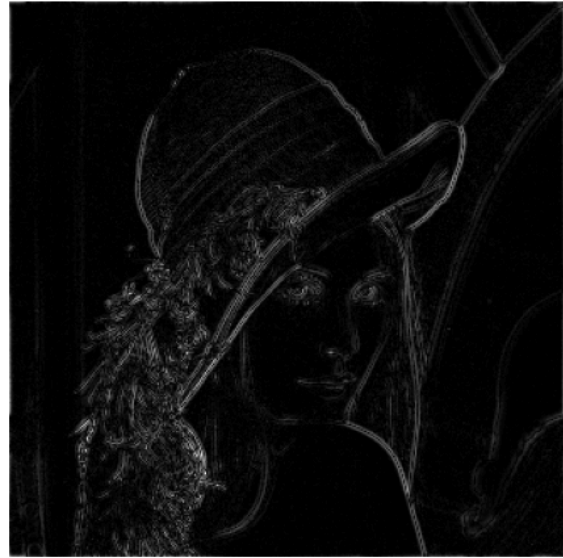
Butterworth High Pass Filter ($n=2$, $D_0=50$)



Original Image



Gaussian High Pass Filter (D0=50)

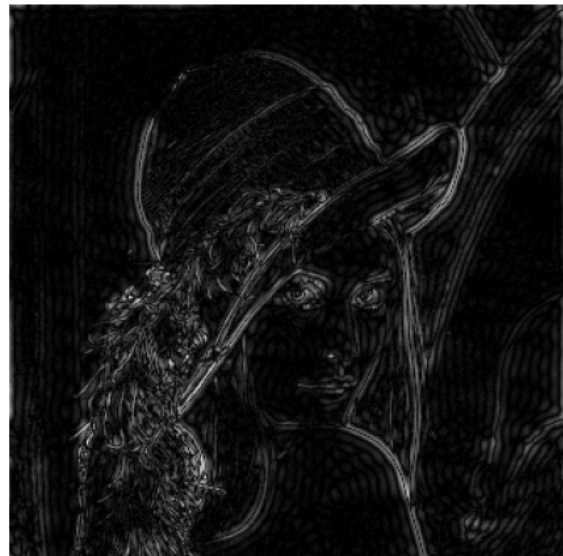


```
# Apply Ideal High-Pass Filter with ringing effect demonstration
cutoff = 30 # Sharp cut-off frequency to highlight ringing
H_ideal_ringing = ideal_high_pass_filter(img.shape, cutoff)
ideal_filtered_img_ringing = apply_filter_and_inverse(f_transform_shifted, H_ideal_ringing)
plot_images(img, ideal_filtered_img_ringing, f"Ideal High Pass Filter - Ringing Effect (D0={cutoff})")
```

Original Image



Ideal High Pass Filter - Ringing Effect (D0=30)



```
# Compare Butterworth High-Pass Filters for different cutoff
frequencies
cutoffs = [5, 15, 30, 90, 120]
n = 2 # Fixed order of the filter

for c in cutoffs:
    H_butterworth = butterworth_high_pass_filter(img.shape, c,
n)

    butterworth_filtered_img =
apply_filter_and_inverse(f_transform_shifted, H_butterworth)
    plot_images(img, butterworth_filtered_img, f"Butterworth
High Pass Filter (n={n}, D0={c})")
```

Original Image



Butterworth High Pass Filter ($n=2$, $D_0=5$)



Original Image



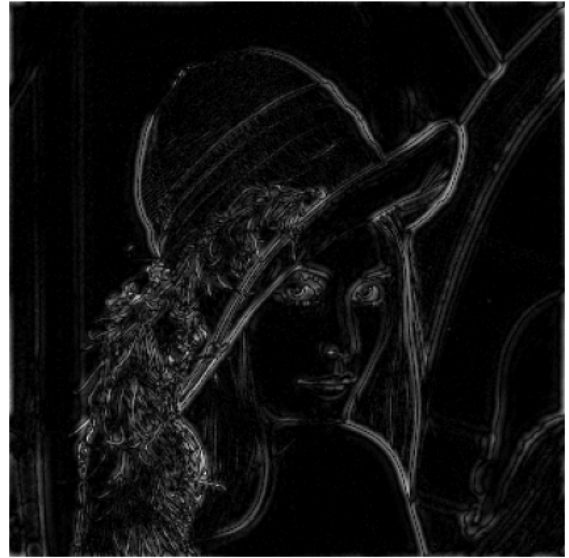
Butterworth High Pass Filter ($n=2$, $D_0=15$)



Original Image



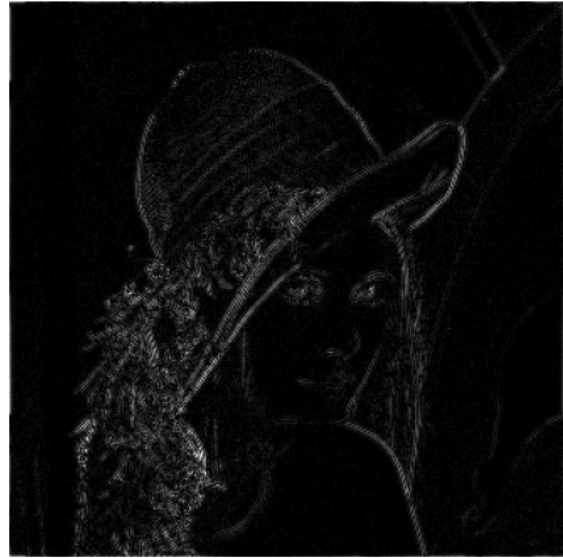
Butterworth High Pass Filter ($n=2$, $D_0=30$)



Original Image



Butterworth High Pass Filter (n=2, D0=90)



Original Image



Butterworth High Pass Filter (n=2, D0=120)



```
# Compare Gaussian High-Pass Filters for different cutoff
frequencies
cutoffs = [5, 15, 30, 90, 120]

for c in cutoffs:
    H_gaussian = gaussian_high_pass_filter(img.shape, c)
    gaussian_filtered_img =
    apply_filter_and_inverse(f_transform_shifted, H_gaussian)
```

```
plot_images(img, gaussian_filtered_img, f"Gaussian High Pass  
Filter (D0={c})")
```

Original Image



Gaussian High Pass Filter (D0=5)



Original Image



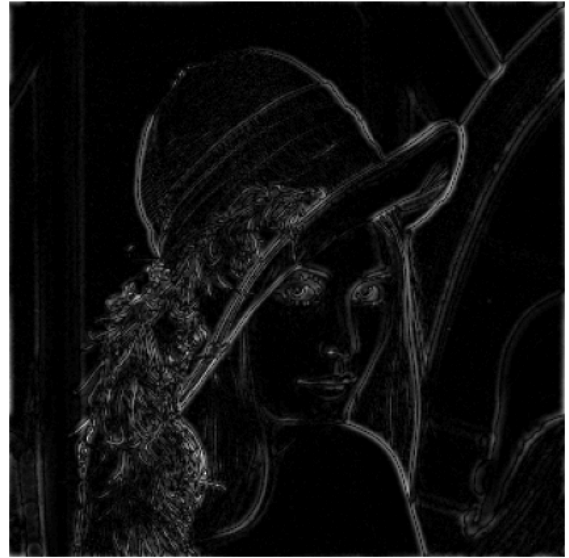
Gaussian High Pass Filter (D0=15)



Original Image



Gaussian High Pass Filter ($D_0=30$)



Original Image



Gaussian High Pass Filter ($D_0=90$)



Original Image



Gaussian High Pass Filter ($D_0=120$)

